

Papers on Compiler Optimizations: Analysis and Transformations (1952-1994)

Federico Bruzzone 
Computer Science Department
Università degli Studi di Milano
federico.bruzzone@unimi.it
<https://federicobruzzone.github.io/>

30 September 2025

Title	Venue Type	Venue Name	Type	Year	Sources
The problem of simplifying truth functions [9]	T&F	AMM	J	1952	O
Minimization of Boolean functions [10]	Bell Labs	Bell System Tech. J.	J	1956	O
An algorithm for translating Boolean expressions [11]	ACM	JACM	J	1962	S
High speed compilation of efficient object code [12]	ACM	CACM	J	1965	D
Peephole optimization [13]	ACM	CACM	J	1965	E
Program optimization [14]	-	PP	C	1966	A, D, E, M, O, S
Index Register Allocation [15]	ACM	JACM	J	1966	E
Analysis of Programs for Parallel Processing [16]	IEEE	TC (TEC)	J	1966	O
Object code optimization [17]	ACM	CACM	J	1969	A, D, E
Expression optimization using unary complement operators [18]	ACM	PLDI (SCO)	C	1970	E
Local optimizations [19]	ACM	PLDI (SCO)	C	1970	E
Detection and parallel execution of independent instructions [20]	IEEE	TCO	J	1970	S
Global common subexpression elimination [21]	ACM	PLDI (SCO)	C	1970	D, E, M
The Generation of Optimal Code for Arithmetic Expressions [22]	ACM	JACM	J	1970	D, E, M
Control flow analysis [23]	ACM	PLDI (SCO)	C	1970	D, E
A Basis for Program Optimization [24]	IFIP	NH	C	1971	A, D, E, M, O, S
A catalogue of optimizing transformations [25]	-	PH	C	1972	A, D, E, M, O, S
Flow graph reducibility [26]	ACM	STOC	C	1972	D
Use-definition chains with applications [27]	Elsevier	COLA (COMLAN)	C	1972	E
A global flow analysis algorithm [28]	T&F	JCM	J	1972	E
Safety of code motion [29]	T&F	JCM	J	1972	O
On the Number of Operations Simultaneously Executable in Fortran-Like Programs and Their Resulting Speedup [30]	IEEE	TCO	J	1972	D, O
Smallest-last ordering and clustering and graph coloring algorithms [31]	ACM	JACM	J	1973	-
Complete register allocation problems [32]	ACM	STOC	C	1973	
Testing flow graph reducibility [33]	ACM	STOC	C	1973	A
A unified approach to global program optimization [34]	ACM	POPL	C	1973	A, D, E, M, O, S
Fast algorithms for the elimination of common subexpressions [35]	Springer	Acta Inf.	J	1973	A, D, E, O

Continued on next page

Title	Venue Type	Venue Name	Type	Year	Sources
Interprocedural Analysis and the Information derived by it [36]	Springer	Prog. Meth.	J	1974	-
Register allocation via usage counts [37]	ACM	CACM	J	1974	A
Analysis of structured programs [38]	ACM	STOC	C	1974	D
Characterizations of Reducible Flow Graphs [39]	ACM	JACM	J	1974	D, E
The parallel execution of DO loops [40]	ACM	CACM	J	1974	D, O
Interprocedural Data Flow Analysis [41]	-	IFIP	J	1974	D, O
Application of lattice algebra to loop optimization [42]	ACM	POPL	C	1975	-
Program optimization - theory and practice [43]	ACM	PLDI (SCC)	C	1975	-
Time and parallel processor bounds for linear recurrence systems [44]	IEEE	TCO	J	1975	S
A simple algorithm for global data flow analysis problems [45]	SIAM	SICOMP	J	1975	A, E, O
Program improvement by source to source transformation [46]	ACM	POPL	C	1976	-
A program data flow analysis procedure [47]	ACM	CACM	J	1976	A
Recursion analysis for compiler optimization [48]	ACM	CACM	J	1976	O
Optimal Code Generation for Expression Trees [49]	ACM	JACM	J	1976	A, D, E
A Fast and Usually Linear Algorithm for Global Flow Analysis [50]	ACM	JACM	J	1976	E, O
Code generation for expressions with common subexpressions [51]	ACM	POPL	C	1976	E, S
Global data flow analysis and iterative algorithms [52]	ACM	JACM	J	1976	E, O
Compiler Analysis of the Value Ranges for Variables [53]	IEEE	TSE	J	1977	-
On Live-Dead Analysis for Global Data Flow Problems [54]	ACM	JACM	J	1977	A
Symbolic evaluation and the global value graph [55]	ACM	POPL	C	1977	A
High-level data flow analysis [56]	ACM	CACM	J	1977	A
Abstract interpretation [57]	ACM	POPL	C	1977	D
An algorithm for reduction of operator strength [58]	ACM	CACM	J	1977	E
A transformation system for developing recursive programs [59]	ACM	JACM	J	1977	S
Arithmetic shifting considered harmful [60]	ACM	SIGPLAN Notices	J	1977	S
Monotone data flow analysis frameworks [61]	Springer	Acta Inf.	J	1977	A, D, E, O
Program Improvement by Source-to-Source Transformation [62]	ACM	CACM	J	1977	D, O
An analysis of inline substitution for a structured programming language [63]	ACM	CACM	J	1977	M, S
Program Optimization Using Invariants [64]	IEEE	TSE	J	1978	-
A new method for compiler code generation [65]	ACM	POPL	C	1978	D
A practical interprocedural data flow analysis algorithm [66]	ACM	CACM	J	1978	A, D, E, O
Data Flow Analysis for Procedural Languages [67]	ACM	JACM	J	1979	A
Constructing the Call Graph of a Program [68]	IEEE	TSE	J	1979	O
Data flow languages [69]	IEEE	MARK	W	1979	S
Time and parallel processor bounds for Fortran-like loops [70]	IEEE	TCO	J	1979	S
Unrolling loops in Fortran [71]	Wiley	SPE	J	1979	S
A fast algorithm for finding dominators in a flowgraph [72]	ACM	TOPLAS	J	1979	A, E, M, O
An efficient way to find the side effects of procedural calls and the aliases of variables [73]	ACM	POPL	C	1979	A, D, E, S
Global optimization by suppression of partial redundancies [74]	ACM	CAMC	J	1979	A, D, E, O, S
Predicting the effects of optimization on a procedure body [75]	ACM	PLDI (SCC)	C	1979	E, S
Structural analysis: A new approach to flow analysis in optimizing compilers [76]	Elsevier	COLA (COMLAN)	C	1980	A
The design and application of a retargetable peephole optimizer [77]	ACM	TOPLAS	J	1980	E
Data flow supercomputers [78]	IEEE	Computer	J	1980	S
Program optimization and parallelization using idioms [79]	ACM	POPL	C	1980	-
High-speed multiprocessors and compilation techniques [80]	IEEE	TCO	J	1980	S
A composite algorithm for strength reduction and code movement optimization [81]	Springer	ACIS	J	1980	E

Continued on next page

Title	Venue Type	Venue Name	Type	Year	Sources
Interprocedural data flow analysis in the presence of pointers, procedure variables, and label variables [82]	ACM	POPL	C	1980	A, E, O
Deciding Linear Inequalities by Computing Loop Residues [83]	ACM	JACM	J	1981	D
A precise inter-procedural data flow algorithm [84]	ACM	POPL	C	1981	O, S
Register allocation via coloring [85]	Elsevier	COLA (COMLAN)	C	1981	A, D, E, O, S
Reduction of operator strength [86]	NJ	Program flow analysis	J	1981	A, E, S
Dependence graphs and compiler optimizations [87]	ACM	POPL	C	1981	O, S
On the performance enhancement of paging systems through program analysis and transformations [88]	IEEE	TCO	J	1981	D, S
Using Peephole Optimization on Intermediate Code [89]	ACM	TOPLAS	J	1982	-
Optimizing delayed branches [90]	ACM	MICRO	W	1982	D
A composite hoisting-strength reduction transformation for global program optimization part i [91]	T&F	JCM	J	1982	E
Optimization of range checking [92]	ACM	PLDI (SCC)	C	1982	E
Register allocation and spilling via graph coloring [93]	ACM	PLDI	C	1982	A, D, E, M, O, S
Experience with the SETL optimizer [94]	ACM	TOPLAS	J	1983	S
Postpass Code Optimization of Pipeline Constraints [95]	ACM	TOPLAS	J	1983	A, E
Conversion of control dependence to data dependence [96]	ACM	POPL	C	1983	O, S
Code selection through object code optimization [97]	ACM	TOPLAS	J	1984	-
Register allocation and exhaustive peephole optimization [98]	Wiley	SPE	J	1984	E
Automatic generation of peephole optimizations [99]	Springer	CC	C	1984	E
Analysis of interprocedural side effects in a parallel programming environment [100]	Springer	ICSP	C	1984	O
Polyvariant mixed computation for analyzer programs [101]	Springer	Acta Inf.	J	1984	O
Stream processing [102]	ACM	LFP	C	1984	O
A hierarchical basis for reordering transformations [103]	ACM	POPL	C	1984	O
Register allocation by priority-based coloring [104]	ACM	PLDI (SCC)	C	1984	A, E, O
Automatic loop interchange [105]	ACM	PLDI (SCC)	C	1984	O, S
Efficient computation of flow insensitive interprocedural summary information [106]	ACM	PLDI (SCC)	C	1984	A, O
Program transformations in a denotational setting [107]	ACM	TOPLAS	J	1985	-
On linearizing parallel code [108]	ACM	POPL	C	1985	O
Distributed execution of functional programs using serial combinators [109]	IEEE	TCO	J	1985	S
Strictness analysis-a practical approach [110]	Springer	FPCA	C	1985	S
A linear algorithm for finding dominators in flow graphs and related problems [111]	ACM	STOC	C	1985	E, M, O
Advanced compiler optimizations for supercomputers [112]	ACM	CACM	J	1986	-
Efficient compilation of linear recursive functions into object level loops [113]	ACM	PLDI (SCC)	C	1986	-
The impact of interprocedural analysis and optimization in the Rn programming environment [114]	ACM	TOPLAS (LOPES)	C	1986	-
Efficient instruction scheduling for a pipelined architecture [115]	ACM	PLDI (SCC)	C	1986	A
Efficient symbolic analysis of programs [116]	ACM	JCSS	J	1986	A
Graph-Based Algorithms for Boolean Function Manipulation [117]	IEEE	TC	J	1986	D
Loops skewing: The wavefront method revisited [118]	Springer	JPP	J	1986	O
Highly concurrent scalar processing [119]	ACM	CAN	J	1986	O
Multiplication by integer constants [120]	Wiley	SPE	J	1986	S
Global register allocation at link time [121]	ACM	PLDI	C	1986	A, E

Continued on next page

Title	Venue Type	Venue Name	Type	Year	Sources
Interprocedural constant propagation [122]	ACM	PLDI	C	1986	A, D, E, O, S
Interprocedural optimization: eliminating unnecessary recompilation [123]	ACM	CC	C	1986	A, E, O
Interprocedural dependence analysis and parallelization [124]	ACM	PLDI	C	1986	O, S
Effectiveness of a machine-level, global optimizer [125],	ACM	PLDI (SCC)	C	1986	A, E
Direct parallelization of call statements [126]	ACM	PLDI (SCC)	C	1986	O, S
Code motion of control structures in high-level languages [127]	ACM	POPL	C	1986	E, M
Automatic inference and fast interpretation of peephole optimization rules [128]	Wiley	SPE	J	1987	E
Compiler Algorithms for Synchronization [129]	IEEE	TCO	J	1987	O
Automatic decomposition of scientific programs for parallel execution [130]	ACM	POPL	C	1987	O
Guided Self-Scheduling: A Practical Scheduling Scheme for Parallel Supercomputers [131]	IEEE	TCO	J	1987	S
Loop coalescing: A compiler transformation for parallel machines [132]	ACM	ICPP	C	1987	S
Strategies for cache and local memory management by global program transformation [133]	Springer	ICS	C	1987	S
The program dependence graph and its use in optimization [134]	ACM	TOPLAS	J	1987	A, E, M, O
Automatic translation of Fortran programs to vector form [135]	ACM	TOPLAS	J	1987	D, O, S
Safety consideration for storage allocation optimizations [136]	ACM	PLDI	C	1988	-
Efficient and correct execution of parallel programs that share memory [137]	ACM	TOPLAS (LOPES)	C	1988	-
On the control dependence in the program dependence graph [138]	ACM	CSC	C	1988	-
Resource requirements of dataflow programs [139]	ACM	ISCA	C	1988	-
Incremental data flow analysis via dominator and attribute update [140]	ACM	POPL	C	1988	-
An efficient approach to data flow analysis in a multiple pass global optimizer [141]	ACM	PLDI	C	1988	A
A solution to a problem with Morel and Renvoise's "Global optimization by suppression of partial redundancies" [142]	ACM	TOPLAS	J	1988	E
Efficient computation of flow-insensitive interprocedural summary information—a correction [143]	ACM	SIGPLAN Notices	J	1988	O
Dependence of multi-dimensional array references [144]	ACM	ICS	C	1988	O
The importance of direct dependences for automatic parallelization [145]	ACM	ICS	C	1988	O
Introducing symbolic problem solving techniques in the dependence testing phases of a vectorizer [146]	ACM	ICS	C	1988	O
Generating sequential code from parallel code [147]	ACM	SC	C	1988	O
CRegs: a new kind of memory for referencing arrays and pointers [148]	ACM	ICS	C	1988	-
Advanced loop optimizations for parallel computers [149]	ACM	ICS	C	1988	S
An introduction to a formal theory of dependence analysis [150]	Springer	JSC	J	1988	S
Analysis of interprocedural side effects in a parallel programming environment [151]	ACM	ICS	C	1988	S
Array expansion [152]	ACM	ICS	C	1988	S
Loop quantization: A generalized loop unwinding technique [153]	ACM	JPDC	J	1988	S
Supernode partitioning [154]	ACM	PLDI	C	1988	S
A fast algorithm for code movement optimisation [155]	ACM	PLDI	C	1988	A, E
Compiling programs for distributed-memory multiprocessors [156]	Elsevier	JSC	J	1988	O, S
Optimal loop parallelization [157]	ACM	PLDI	C	1988	A, E, M, S
Software pipelining: an effective scheduling technique for VLIW machines [158]	ACM	PLDI	C	1988	D, E

Continued on next page

Title	Venue Type	Venue Name	Type	Year	Sources
The program summary graph and flow-sensitive interprocedural data flow analysis [159]	ACM	PLDI	C	1988	A, O
A framework for determining useful parallelism [160]	ACM	ICS	C	1988	O, S
An overview of the PTRAN analysis system for multiprocessing [161]	Elsevier	JPDC	J	1988	D, S
Efficient interprocedural analysis for program parallelization and restructuring [162]	ACM	PPoPP (PPEALS)	C	1988	O, S
Compiling issues for supercomputers [163]	ACM/IEE	SC	C	1988	O, S
Minimizing register usage penalty at procedure calls [164]	ACM	PLDI	C	1988	A, S
Estimating interlock and improving balance for pipelined architectures [165]	Elsevier	JPDC	J	1988	O, S
Global value numbers and redundant computations [166]	ACM	POPL	C	1988	E, O
Perfect pipelining: A new loop parallelization technique [167]	Springer	ESOP	C	1988	A, S
Code scheduling and register allocation in large basic blocks [168]	ACM	SC	C	1988	A, E
Interprocedural side-effect analysis in linear time [169]	ACM	PLDI	C	1988	A, E, O
A technique for summarizing data access and its use in parallelism enhancing transformations [170]	ACM	PLDI	C	1989	-
Interprocedural analysis vs. procedure integration [171]	Elsevier	IPL	J	1989	-
Unified management of registers and cache using liveness and cache bypass [172]	ACM	PLDI	C	1989	A
A new algorithm for composite hoisting and strength reduction optimisation [173]	T&F	JCM	J	1989	E
Interprocedural data flow testing [174]	ACM	SEN	C	1989	-
Program optimization for instruction caches [175]	ACM	ASPLOS	C	1989	A
Dependence analysis for pointer variables [176]	ACM	PLDI	C	1989	E
The program dependence graph and vectorization [177]	ACM	POPL	C	1989	O
Achieving high instruction cache performance with an optimizing compiler [178]	ACM	ISCA	C	1989	S
Evaluating the performance of four snooping cache coherency protocols [179]	IEEE	ICSA	C	1989	S
Scans as primitive parallel operations [180]	IEEE	TCO	J	1989	S
Code generation using tree matching and dynamic programming [181]	ACM	TOPLAS	J	1989	A, D, E, M
Register allocation via clique separators [182]	ACM	PLDI	C	1989	A, E
Fast interprocedural alias analysis [183]	ACM	POPL	C	1989	A, E, O, S
Coloring heuristics for register allocation [184]	ACM	PLDI	C	1989	A, E, O
More iteration space tiling [185]	ACM/IEE	SC	C	1989	A, S
Data dependence analysis on multi-dimensional array references [186]	ACM	ICS	C	1989	O, S
Spill code minimization techniques for optimizing compilers [187]	ACM	PLDI	C	1989	A, E
Customization: Optimizing compiler technology for SELF, a dynamically-typed OOP language [188]	ACM	PLDI	C	1989	O, S
An efficient method of computing static single assignment form [189]	ACM	POPL	C	1989	A, O
The value flow graph: A program representation for optimal program transformations [190]	Springer	ESOP	C	1990	-
Compiler techniques for data partitioning of sequentially iterated parallel loops [191]	ACM	SC	C	1990	-
Vectorization of tree traversals [192]	Elsevier	JCP	J	1990	-
Loop displacement: an approach for transforming and scheduling loops for parallel execution [193]	ACM	SC	C	1990	-
Experience with interprocedural analysis of array side effects [194]	ACM	SC	C	1990	-
An approach to ordering optimizing transformations [195]	ACM	PPoPP	C	1990	A
Register allocation across procedure and module boundaries [196]	ACM	PLDI	C	1990	A
Region Scheduling: An Approach for Detecting and Redistributing Parallelism [197]	IEEE	TSE	J	1990	E

Continued on next page

Title	Venue Type	Venue Name	Type	Year	Sources
Constructing the procedure call multigraph [198]	IEEE	TSE	J	1990	O
On the perfect accuracy of an approximate subscript analysis test [199]	ACM	CAN	J	1990	O
Structured dataflow analysis for arrays and its use in an optimizing compiler [200]	Wiley	Software: Practice and Experience	J	1990	O
Compilation of Haskell array comprehensions for scientific computing [201]	ACM	PLDI	C	1990	S
How to read floating point numbers accurately [202]	ACM	PLDI	C	1990	S
How to print floating-point numbers accurately [203]	ACM	PLDI	C	1990	S
Profile guided code positioning [204]	ACM	PLDI	C	1990	S
Updating distributed variables in local computations [205]	Wiley	SPE	J	1990	S
An interval-based approach to exhaustive and incremental interprocedural data-flow analysis [206]	ACM	TOPLAS	J	1990	E, O
The priority-based coloring approach to register allocation [207]	ACM	TOPLAS	J	1990	A, D, E, S
Improving register allocation for subscripted variables [208]	ACM	PLDI	C	1990	A, E, M, O, S
Analysis of pointers and structures [209]	ACM	PLDI	C	1990	A, E
Loop distribution with arbitrary control flow [210]	ACM	PLDI	C	1990	O, S
Graph coloring register allocation for processors with multi-register operands [211]	ACM	PLDI	C	1990	A, E
A data locality optimizing algorithm [212]	ACM	PLDI	C	1991	-
On optimal parallelization of arbitrary loops [213]	Elsevier	JPDC	J	1991	-
On the adequacy of dependence-based representations for programs with heaps [214]	Springer	TACS	C	1991	-
Automatic construction of sparse data flow evaluation graphs [215]	ACM	POPL	C	1991	-
Compiler optimizations for Fortran D on MIMD distributed-memory machines [216]	ACM	SC	C	1991	-
Register allocation via hierarchical graph coloring [217]	ACM	PLDI	C	1991	A
Circular scheduling: a new technique to perform software pipelining [218]	ACM	PLDI	C	1991	A
Efficient DAG construction and heuristic calculation for instruction scheduling [219]	ACM	MICRO	W	1991	E
Efficiently computing static single assignment form and the control dependence graph [220]	ACM	TOPLAS	J	1991	M
Software prefetching [221]	ACM	ASPLOS	C	1991	O
Compiling global name-space parallel loops for distributed execution [222]	IEEE	TPDS	J	1991	O
An implementation of interprocedural bounded regular section analysis [223]	ACM	TPDS	J	1991	O
Limits of instruction-level parallelism [224]	ACM	ASPLOS	C	1991	S
Uniform techniques for loop optimization [225]	ACM	ICS	C	1991	S
A data locality optimizing [212]	ACM	PLDI	C	1991	A, D, M, O
Constant propagation with conditional branches [226]	ACM	TOPLAS	J	1991	A, E, M, O, S
Efficient and exact data dependence analysis [227]	ACM	PLDI	C	1991	A, D, O
Efficiently computing static single assignment [228]	ACM	TOPLAS	J	1991	A, D, E, O
Global instruction scheduling for superscalar machines [229]	ACM	PLDI	C	1991	A, D, E
Practical adaption of the global optimization algorithm of Morel and Renvoise [230]	ACM	TOPLAS	J	1991	A, E
The cache performance and optimizations of blocked algorithms [231]	ACM	ASPLOS	C	1991	A, D
A loop transformation theory and an algorithm to maximize parallelism [232]	IEEE	TPDS	J	1991	O, S

Continued on next page

Title	Venue Type	Venue Name	Type	Year	Sources
Dataflow analysis of array and scalar references [233]	Springer	JPP	J	1991	A, S
Optimization of array accesses by collective loop transformations [234]	ACM	SC	C	1991	D, O
Interprocedural transformations for parallel code generation [235]	ACM/IEE	SC	C	1991	O, S
Practical dependence testing [236]	ACM	PLDI	C	1991	A, O
Procedure merging with instruction caches [237]	ACM	PLDI	C	1991	A, S
An experiment with inline substitution [238]	Wiley	SPE	J	1991	E, S
Access normalization: loop restructuring for NUMA compilers [?]]	ACM	ASPLOS	C	1991	-
Non-unimodular transformations of nested loops [239]	ACM	SC	C	1992	-
A general framework for iteration-reordering loop transformations [240]	ACM	PLDI	C	1992	-
A program integration algorithm that accommodates semantics-preserving transformations [241]	ACM	TOSEM	J	1992	-
The transitive closure of control dependence: The iterated join [242]	ACM	TOPLAS (LOPLAS)	J	1992	-
Abstract description of pointer data structures: an approach for improving the analysis and optimization of imperative programs [243]	ACM	TOPLAS (LOPLAS)	J	1992	-
Abstractions for recursive pointer data structures: improving the analysis and transformation of imperative programs [244]	ACM	PLDI	C	1992	-
Generalized dominators and post-dominators [245]	ACM	POPL	C	1992	-
A comprehensive approach to parallel data flow analysis [246]	ACM	ICS	C	1992	-
Integrating scalar optimization and parallelization [247]	Springer	LCPC	C	1992	A
Sharlit—a tool for building optimizers [248]	ACM	PLDI	C	1992	A
How to analyze large programs efficiently and informatively [249]	ACM	PLDI	C	1992	A
Compiler code transformations for superscalar-based high performance systems [250]	IEEE	SC	C	1992	A
Engineering a simple, efficient code-generator generator [251]	ACM	TOPLAS (LOPLAS)	J	1992	D
Some efficient solutions to the affine scheduling problem. I. One-dimensional time [252]	Elsevier	JPP	J	1992	D
Avoiding unconditional jumps by code replication [253]	ACM	PLDI	C	1992	E
Rematerialization [254]	ACM	PLDI	C	1992	E
Coloring register pairs [255]	ACM	TOPLAS (LOPLAS)	J	1992	E
Optimizing for parallelism and data locality [256]	ACM	SC	C	1992	O
Maximizing loop parallelism and improving data locality via loop fusion and distribution [257]	ACM	LCPC	W	1992	O
Beyond induction variables [258]	ACM	PLDI	C	1992	O
Efficient call graph analysis [259]	ACM	TOPLAS (LOPLAS)	J	1992	O
A safe approximate algorithm for interprocedural aliasing [260]	ACM	PLDI	C	1992	O
Relaxing SIMD control flow constraints using loop transformations [244]	ACM	PLDI	C	1992	S
Unexpected side effects of inline substitution: a case study [261]	ACM	TOPLAS (LOPLAS)	J	1992	S
Eliminating false positives using the omega test [262]	ACM	PLDI	C	1992	A, D, O
Lazy code motion [263]	ACM	PLDI	C	1992	A, D, E
Software support for speculative loads [264]	ACM	ASPLOS	C	1992	A, E
A practical algorithm for exact array dependence analysis [265]	ACM	CACM	J	1992	O, S
Array privatization for parallel execution of loops [266]	ACM	ICS	C	1992	O, S
The power test for data dependence [267]	IEEE	TCO	J	1992	A, S
Eliminating branches using a superoptimizer and the GNU C compiler [268]	ACM	PLDI	C	1992	E, S

Continued on next page

Title	Venue Type	Venue Name	Type	Year	Sources
Design and evaluation of a compiler algorithm for prefetching [269]	ACM	ASPLOS	C	1992	D, O
Data dependence and data-flow analysis of arrays [270]	Springer	LCPC	C	1992	-
Advanced compiler optimizations for sparse computations [271]	ACM	SC	C	1993	-
Access normalization: loop restructuring for NUMA computers [272]	ACM	TOCS	J	1993	-
On the conversion of indirect to direct recursion [273]	ACM	TOPLAS (LOPLAS)	J	1993	-
A practical data flow framework for array reference analysis and its use in optimizations [274]	ACM	PLDI	C	1993	-
Using lifetime predictors to improve memory allocation performance [275]	ACM	PLDI	C	1993	-
Data flow analysis for parallel programs [276]	ACM	CSC	C	1993	-
Dependence-based program analysis [277]	ACM	PLDI	C	1993	A
A variation of Knoop, Rüthing, and Steffen's lazy code motion [278]	ACM	PLDI	C	1993	E
Lazy strength reduction [279]	C&H	JPL	J	1993	A, E
Collective loop fusion for array contraction [280]	Springer	LCPC	C	1993	O
Interprocedural constant propagation: an empirical study [281]	ACM	TOPLAS (LOPLAS)	J	1993	O
Symbolic analysis: A basis for parallelization, optimization, and scheduling of programs [282]	Springer	LCPC	W	1993	O
Array-data flow analysis and its use in array privatization [283]	ACM	POPL	C	1993	S
Automatic array alignment in data-parallel programs [284]	ACM	POPL	C	1993	S
Global optimizations for parallelism and locality on scalable parallel machines [285]	ACM	PLDI	C	1993	S
Instruction-Level Parallel Processing: History, Overview, and Perspective [286]	Springer	JSC	J	1993	S
A practical system for intermodule code optimization at link-time [287]	C&H	JPL	J	1993	A
Loop-level parallelism in numeric and symbolic programs [288]	IEEE	TPDS	J	1993	S
Orchestrating interactions among parallel computations [289]	ACM	PLDI	C	1993	S
The superblock: an effective technique for VLIW and superscalar compilation [290]	Springer	JSC	J	1993	S
Optimizing array bound checks using flow analysis [291]	ACM	PLDI	C	1993	A, E
Register allocation with instruction scheduling [292]	ACM	PLDI	C	1993	A, E
A methodology for procedure cloning [293]	Elsevier	COLA (COMLAN)	J	1993	A, O, S
Efficient flow-sensitive interprocedural computation of pointer-induced aliases and side effects [294]	ACM	POPL	C	1993	E, O, S
Automatic array privatization [295]	Springer	LCPC	W	1993	O, S
Interprocedural modification side effect analysis with pointer aliasing [296]	ACM	PLDI	C	1993	M, S
Branch prediction for free [297]	ACM	PLDI	C	1993	A, M
Enabling unimodular transformations [298]	ACM	SC	C	1994	-
Compiler optimizations for improving data locality [299]	ACM	ASPLOS	C	1994	-
The revival transformation [300]	ACM	PLDI	C	1994	-
The range test: a dependence test for symbolic, non-linear expressions [301]	ACM	SC	C	1994	-
Link-time optimization of address calculation on a 64-bit architecture [302]	ACM	PLDI	C	1994	A
Effective partial redundancy elimination [303]	ACM	PLDI	C	1994	A
Partial dead code elimination [304]	ACM	PLDI	C	1994	A
A general data dependence test for dynamic, pointer-based data structures [243]	ACM	PLDI	C	1994	A
Instruction scheduling over regions: A framework for scheduling across basic blocks [305]	Springer	CC	C	1994	A

Continued on next page

REFERENCES

Title	Venue Type	Venue Name	Type	Year	Sources
Value dependence graphs: representation without taxation [306]	ACM	POPL	C	1994	A
Zero-cost range splitting [307]	ACM	PLDI	C	1994	E
Reducing branch costs via branch alignment [308]	ACM	OSR	J	1994	E
Improving the ratio of memory operations to floating-point operations in loops [309]	ACM	TOPLAS	J	1994	M
Optimizing multi-method dispatch using compressed dispatch tables [310]	ACM	OOPSLA	C	1994	M
Improving the accuracy of static branch prediction using branch correlation [311]	ACM	ASPLOS	C	1994	M
A compiler framework for restructuring data declarations to enhance cache and TLB effectiveness [312]	IEEE	CASCON	C	1994	S
False sharing and spatial locality in multiprocessor caches [313]	IEEE	TCO	J	1994	S
Reassociation and strength reduction [314]	ACM	SCO	C	1994	S
The alignment-distribution graph [315]	Springer	LCPC	W	1994	S
Improvements to graph coloring register allocation [316]	ACM	TOPLAS	J	1994	A, E, M
Context-sensitive interprocedural points-to analysis in the presence of function pointers [317]	ACM	PLDI	C	1994	D, E
Interprocedural may-alias analysis for pointers: beyond k-limiting [318]	ACM	PLDI	C	1994	A, E
Scalar replacement in the presence of conditional control flow [319]	Wiley	SPE	J	1994	E, O

Table 1: The papers from *Advanced Compiler Design and Implementation* [1] (A), *Compilers: Principles, Techniques, and Tools* [2] (D), *Engineering a Compiler* [3] (E), *Modern Compiler Implementation in C/Java/ML* [4–6] (M), *Optimizing Compilers for Modern Architectures: A Dependence-Based Approach* [7] (O), and the Bacon’s survey [8] (S). In the **Type** column, C stands for conference, J for journal, and W for workshop.

References

- [1] Steven Muchnick. *Advanced compiler design implementation*. Morgan kaufmann, 1997.
- [2] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, Boston, MA, USA, second edition, 2006.
- [3] Keith D. Cooper and Linda Torczon. *Engineering a Compiler*. Morgan Kaufmann, November 2022.
- [4] Andrew W Appel. Modern compiler design in c, 1998.
- [5] W Appel Andrew and Palsberg Jens. Modern compiler implementation in java. In *ISBN 0–521–58388–8*. Cambridge University Press, 2002.
- [6] Andrew W Appel. *Modern compiler implementation in ML*. Cambridge university press, 1998.
- [7] Ken Kennedy and John R Allen. *Optimizing compilers for modern architectures: a dependence-based approach*. Morgan Kaufmann Publishers Inc., 2001.
- [8] David F Bacon, Susan L Graham, and Oliver J Sharp. Compiler transformations for high-performance computing. *ACM Computing Surveys (CSUR)*, 26(4):345–420, 1994.
- [9] Willard V Quine. The problem of simplifying truth functions. *The American mathematical monthly*, 59(8):521–531, 1952.
- [10] Edward J McCluskey. Minimization of boolean functions. *The Bell System Technical Journal*, 35(6):1417–1444, 1956.

REFERENCES

- [11] Bruce W Arden, Bernard A Galler, and Robert M Graham. An algorithm for translating boolean expressions. *Journal of the ACM (JACM)*, 9(2):222–239, 1962.
- [12] C. W. Gear. High speed compilation of efficient object code. *Commun. ACM*, 8(8):483–488, August 1965.
- [13] W. M. McKeeman. Peephole optimization. *Commun. ACM*, 8(7):443–444, July 1965.
- [14] Frances E. Allen. Program optimization. In Mark Halpern and Christopher Shaw, editors, *Annual Review of Automatic Programming*, volume 5, pages 239–307. Pergamon Press, Elmsford, NY, 1966.
- [15] L. P. Horwitz, R. M. Karp, R. E. Miller, and S. Winograd. Index register allocation. *J. ACM*, 13(1):43–61, January 1966.
- [16] A. J. Bernstein. Analysis of programs for parallel processing. *IEEE Transactions on Electronic Computers*, EC-15(5):757–763, 1966.
- [17] Edward S. Lowry and C. W. Medlock. Object code optimization. *Commun. ACM*, 12(1):13–22, January 1969.
- [18] Dennis J. Frailey. Expression optimization using unary complement operators. In *Proceedings of a symposium on Compiler optimization* -, page 67–85. ACM Press, 1970.
- [19] John T Bagwell Jr. Local optimizations. In *Proceedings of a symposium on Compiler optimization*, pages 52–66, 1970.
- [20] Garold S Tjaden and Michael J Flynn. Detection and parallel execution of independent instructions. *IEEE Transactions on computers*, 19(10):889–895, 1970.
- [21] John Cocke. Global common subexpression elimination. In *Proceedings of a Symposium on Compiler Optimization*, page 20–24, New York, NY, USA, 1970. Association for Computing Machinery.
- [22] Ravi Sethi and J. D. Ullman. The generation of optimal code for arithmetic expressions. *J. ACM*, 17(4):715–728, October 1970.
- [23] Frances E. Allen. Control flow analysis. *SIGPLAN Not.*, 5(7):1–19, July 1970.
- [24] Frances E. Allen. A basis for program optimization. In Charles V. Freiman, John E. Griffith, and Jack L. Rosenfeld, editors, *Information Processing, Proceedings of IFIP Congress 1971, Volume 1 - Foundations and Systems, Ljubljana, Yugoslavia, August 23-28, 1971*, pages 385–390. North-Holland, 1971.
- [25] Frances E. Allen and John Cocke. A catalogue of optimizing transformations. In Randall Rustin, editor, *Design and Optimization of Compilers*. Prentice-Hall, 1972.
- [26] Matthew S. Hecht and Jeffrey D. Ullman. Flow graph reducibility. In *Proceedings of the Fourth Annual ACM Symposium on Theory of Computing*, STOC '72, page 238–250, New York, NY, USA, 1972. Association for Computing Machinery.
- [27] Ken Kennedy. Use-definition chains with applications. *Computer Languages*, 3(3):163–179, 1978.
- [28] Ken Kennedy. A global flow analysis algorithm. *International Journal of Computer Mathematics*, 3(1-4):5–15, 1972.
- [29] Ken Kennedy. Safety of code motion. *International Journal of Computer Mathematics*, 3(1-4):117–130, 1972.
- [30] D.J. Kuck, Y. Muraoka, and Shyh-Ching Chen. On the number of operations simultaneously executable in fortran-like programs and their resulting speedup. *IEEE Transactions on Computers*, C-21(12):1293–1310, 1972.
- [31] David W. Matula and Leland L. Beck. Smallest-last ordering and clustering and graph coloring algorithms. *J. ACM*, 30(3):417–427, July 1983.
- [32] Ravi Sethi. Complete register allocation problems. In *Proceedings of the Fifth Annual ACM Symposium on Theory of Computing*, STOC '73, page 182–195, New York, NY, USA, 1973. Association for Computing Machinery.

REFERENCES

- [33] Robert Tarjan. Testing flow graph reducibility. In *Proceedings of the Fifth Annual ACM Symposium on Theory of Computing*, STOC '73, page 96–107, New York, NY, USA, 1973. Association for Computing Machinery.
- [34] Gary A Kildall. A unified approach to global program optimization. In *Proceedings of the 1st annual ACM SIGACT-SIGPLAN symposium on Principles of programming languages*, pages 194–206, 1973.
- [35] Jeffrey D Ullman. Fast algorithms for the elimination of common subexpressions. *Acta Informatica*, 2(3):191–213, 1973.
- [36] Frances E. Allen. Interprocedural analysis and the information derived by it. In *Programming Methodology, 4th Informatik Symposium*, page 291–322, Berlin, Heidelberg, 1974. Springer-Verlag.
- [37] Robert A Freiburghouse. Register allocation via usage counts. *Communications of the ACM*, 17(11):638–642, 1974.
- [38] S. Rao Kosaraju. Analysis of structured programs. In *Proceedings of the Fifth Annual ACM Symposium on Theory of Computing*, STOC '73, page 240–252, New York, NY, USA, 1973. Association for Computing Machinery.
- [39] M. S. Hecht and J. D. Ullman. Characterizations of reducible flow graphs. *J. ACM*, 21(3):367–375, July 1974.
- [40] Leslie Lamport. The parallel execution of do loops. *Commun. ACM*, 17(2):83–93, February 1974.
- [41] Frances E. Allen. Interprocedural data flow analysis. In Jack L. Rosenfeld, editor, *Information Processing, Proceedings of the 6th IFIP Congress 1974, Stockholm, Sweden, August 5-10, 1974*, pages 398–402. North-Holland, 1974.
- [42] Amelia Fong, John Kam, and Jeffrey Ullman. Application of lattice algebra to loop optimization. In *Proceedings of the 2nd ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, POPL '75, page 1–9, New York, NY, USA, 1975. Association for Computing Machinery.
- [43] David B. Loveman and Ross A. Faneuf. Program optimization - theory and practice. *SIGPLAN Not.*, 10(3):97–102, January 1975.
- [44] Shyh-Ching Chen and David J. Kuck. Time and parallel processor bounds for linear recurrence systems. *IEEE Transactions on Computers*, 100(7):701–717, 1975.
- [45] Matthew S Hecht and Jeffrey D Ullman. A simple algorithm for global data flow analysis problems. *SIAM Journal on Computing*, 4(4):519–532, 1975.
- [46] David B. Loveman. Program improvement by source to source transformation. In *Proceedings of the 3rd ACM SIGACT-SIGPLAN Symposium on Principles on Programming Languages*, POPL '76, page 140–152, New York, NY, USA, 1976. Association for Computing Machinery.
- [47] F. E. Allen and J. Cocke. A program data flow analysis procedure. *Commun. ACM*, 19(3):137, March 1976.
- [48] Kenneth G Walter. Recursion analysis for compiler optimization. *Communications of the ACM*, 19(9):514–516, 1976.
- [49] A. V. Aho and S. C. Johnson. Optimal code generation for expression trees. *J. ACM*, 23(3):488–501, July 1976.
- [50] Susan L. Graham and Mark Wegman. A fast and usually linear algorithm for global flow analysis. *J. ACM*, 23(1):172–202, January 1976.
- [51] Alfred V Aho, Stephen C Johnson, and Jeffrey D Ullman. Code generation for expressions with common subexpressions. In *Proceedings of the 3rd ACM SIGACT-SIGPLAN symposium on Principles on programming languages*, pages 19–31, 1976.
- [52] John B Kam and Jeffrey D Ullman. Global data flow analysis and iterative algorithms. *Journal of the ACM (JACM)*, 23(1):158–171, 1976.

REFERENCES

- [53] W.H. Harrison. Compiler analysis of the value ranges for variables. *IEEE Transactions on Software Engineering*, SE-3(3):243–250, 1977.
- [54] Lawrence T. Kou. On live-dead analysis for global data flow problems. *J. ACM*, 24(3):473–483, July 1977.
- [55] John H. Reif and Harry R. Lewis. Symbolic evaluation and the global value graph. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, POPL ’77, page 104–118, New York, NY, USA, 1977. Association for Computing Machinery.
- [56] Barry K. Rosen. High-level data flow analysis. *Commun. ACM*, 20(10):712–724, October 1977.
- [57] Patrick Cousot and Radhia Cousot. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, POPL ’77, page 238–252, New York, NY, USA, 1977. Association for Computing Machinery.
- [58] John Cocke and Ken Kennedy. An algorithm for reduction of operator strength. *Commun. ACM*, 20(11):850–856, November 1977.
- [59] Rod M Burstall and John Darlington. A transformation system for developing recursive programs. *Journal of the ACM (JACM)*, 24(1):44–67, 1977.
- [60] Guy L. Steele. Arithmetic shifting considered harmful. *SIGPLAN Not.*, 12(11):61–69, November 1977.
- [61] John B Kam and Jeffrey D Ullman. Monotone data flow analysis frameworks. *Acta informatica*, 7(3):305–317, 1977.
- [62] David B. Loveman. Program Improvement by Source-to-Source Transformation. *Journal of the ACM*, 24(1):121–145, January 1977.
- [63] Robert W. Scheifler. An analysis of inline substitution for a structured programming language. *Commun. ACM*, 20(9):647–654, September 1977.
- [64] S. Katz. Program optimization using invariants. *IEEE Transactions on Software Engineering*, SE-4(5):378–389, 1978.
- [65] R. Steven Glanville and Susan L. Graham. A new method for compiler code generation. In *Proceedings of the 5th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, POPL ’78, page 231–254, New York, NY, USA, 1978. Association for Computing Machinery.
- [66] Jeffrey M. Barth. A practical interprocedural data flow analysis algorithm. *Commun. ACM*, 21(9):724–736, September 1978.
- [67] Barry K. Rosen. Data flow analysis for procedural languages. *J. ACM*, 26(2):322–344, April 1979.
- [68] B.G. Ryder. Constructing the call graph of a program. *IEEE Transactions on Software Engineering*, SE-5(3):216–226, 1979.
- [69] William B Ackerman. Data flow languages. In *1979 International Workshop on Managing Requirements Knowledge (MARK)*, pages 1087–1095. IEEE, 1979.
- [70] Shyh-Ching Chen et al. Time and parallel processor bounds for fortran-like loops. *IEEE Transactions on Computers*, 100(9):660–670, 1979.
- [71] Jack J Dongarra and A_R Hinds. Unrolling loops in fortran. *Software: Practice and Experience*, 9(3):219–226, 1979.
- [72] Thomas Lengauer and Robert Endre Tarjan. A fast algorithm for finding dominators in a flowgraph. *ACM Trans. Program. Lang. Syst.*, 1(1):121–141, January 1979.
- [73] John P Banning. An efficient way to find the side effects of procedure calls and the aliases of variables. In *Proceedings of the 6th ACM SIGACT-SIGPLAN symposium on Principles of programming languages*, pages 29–41, 1979.
- [74] E. Morel and C. Renvoise. Global optimization by suppression of partial redundancies. *Commun.*

REFERENCES

- ACM*, 22(2):96–103, February 1979.
- [75] J Eugene Ball. Predicting the effects of optimization on a procedure body. *ACM SIGPLAN Notices*, 14(8):214–220, 1979.
 - [76] M. Sharir. Structural analysis: A new approach to flow analysis in optimizing compilers. *Comput. Lang.*, 5(3–4):141–153, January 1980.
 - [77] Jack W Davidson and Christopher W Fraser. The design and application of a retargetable peephole optimizer. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 2(2):191–202, 1980.
 - [78] Jack B Dennis. Data flow supercomputers. *Computer*, 13(11):48–56, 1980.
 - [79] Shlomit S. Pinter and Ron Y. Pinter. Program optimization and parallelization using idioms. In *Proceedings of the 18th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL ’91, page 79–92, New York, NY, USA, 1991. Association for Computing Machinery.
 - [80] Padua, Kuck, and Lawrie. High-speed multiprocessors and compilation techniques. *IEEE Transactions on Computers*, 100(9):763–776, 1980.
 - [81] Dhananjay M Dhamdhere and JR Isaac. A composite algorithm for strength reduction and code movement optimization. *International Journal of Computer & Information Sciences*, 9:243–273, 1980.
 - [82] William E. Weihl. Interprocedural data flow analysis in the presence of pointers, procedure variables, and label variables. In *Proceedings of the 7th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL ’80, page 83–94, New York, NY, USA, 1980. Association for Computing Machinery.
 - [83] Robert Shostak. Deciding linear inequalities by computing loop residues. *J. ACM*, 28(4):769–779, October 1981.
 - [84] Eugene M. Myers. A precise inter-procedural data flow algorithm. In *Proceedings of the 8th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL ’81, page 219–230, New York, NY, USA, 1981. Association for Computing Machinery.
 - [85] Gregory J Chaitin, Marc A Auslander, Ashok K Chandra, John Cocke, Martin E Hopkins, and Peter W Markstein. Register allocation via coloring. *Computer languages*, 6(1):47–57, 1981.
 - [86] Frances E Allen, John Cocke, and Ken Kennedy. Reduction of operator strength. *Program Flow Analysis*, pages 79–101, 1981.
 - [87] D. J. Kuck, R. H. Kuhn, D. A. Padua, B. Leasure, and M. Wolfe. Dependence graphs and compiler optimizations. In *Proceedings of the 8th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL ’81, page 207–218, New York, NY, USA, 1981. Association for Computing Machinery.
 - [88] Kuck and Lawrie. On the performance enhancement of paging systems through program analysis and transformations. *IEEE Transactions on Computers*, 100(5):341–356, 1981.
 - [89] Andrew S. Tanenbaum, Hans van Staveren, and Johan W. Stevenson. Using peephole optimization on intermediate code. *ACM Trans. Program. Lang. Syst.*, 4(1):21–36, January 1982.
 - [90] Thomas R. Gross and John L. Hennessy. Optimizing delayed branches. *SIGMICRO Newsl.*, 13(4):114–120, October 1982.
 - [91] SM Joshi and Dhananjay M Dhamdhere. A composite hoisting-strength reduction transformation for global program optimization part i. *International Journal of Computer Mathematics*, 11(1):21–41, 1982.
 - [92] Victoria Markstein, John Cocke, and Peter Markstein. Optimization of range checking. In *Proceedings of the 1982 SIGPLAN Symposium on Compiler Construction*, SIGPLAN ’82, page 114–119, New York, NY, USA, 1982. Association for Computing Machinery.
 - [93] Gregory J Chaitin. Register allocation and spilling via graph coloring. *ACM Sigplan Notices*, 17(6):98–

REFERENCES

- 101, 1982.
- [94] Stefan M Freudenberger, Jacob T Schwartz, and Micha Sharir. Experience with the setl optimizer. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 5(1):26–45, 1983.
- [95] John L. Hennessy and Thomas Gross. Postpass code optimization of pipeline constraints. *ACM Trans. Program. Lang. Syst.*, 5(3):422–448, July 1983.
- [96] John R Allen, Ken Kennedy, Carrie Porterfield, and Joe Warren. Conversion of control dependence to data dependence. In *Proceedings of the 10th ACM SIGACT-SIGPLAN symposium on Principles of programming languages*, pages 177–189, 1983.
- [97] Jack W. Davidson and Christopher W. Fraser. Code selection through object code optimization. *ACM Trans. Program. Lang. Syst.*, 6(4):505–526, October 1984.
- [98] Jack W Davidson and Christopher W Fraser. Register allocation and exhaustive peephole optimization. *Software: Practice and Experience*, 14(9):857–865, 1984.
- [99] Jack W Davidson and Christopher W Fraser. Automatic generation of peephole optimizations. In *Proceedings of the 1984 SIGPLAN symposium on Compiler construction*, pages 111–116, 1984.
- [100] David Callahan and Ken Kennedy. Analysis of interprocedural side effects in a parallel programming environment. In *International Conference on Supercomputing*, pages 138–171. Springer, 1987.
- [101] Mikhail A. Bulyonkov. Polyvariant mixed computation for analyzer programs. *Acta Informatica*, 21(5):473–484, 1984.
- [102] Allen Goldberg and Robert Paige. Stream processing. In *Proceedings of the 1984 ACM Symposium on LISP and Functional Programming*, LFP ’84, page 53–62, New York, NY, USA, 1984. Association for Computing Machinery.
- [103] Joe Warren. A hierarchical basis for reordering transformations. In *Proceedings of the 11th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, POPL ’84, page 272–282, New York, NY, USA, 1984. Association for Computing Machinery.
- [104] Frederick Chow and John Hennessy. Register allocation by priority-based coloring. *SIGPLAN Not.*, 19(6):222–232, June 1984.
- [105] John R Allen and Ken Kennedy. Automatic loop interchange. In *Proceedings of the 1984 SIGPLAN symposium on Compiler construction*, pages 233–246, 1984.
- [106] Keith D Cooper and Ken Kennedy. Efficient computation of flow insensitive interprocedural summary information. In *Proceedings of the 1984 SIGPLAN symposium on Compiler construction*, pages 247–258, 1984.
- [107] Flemming Nielson. Program transformations in a denotational setting. *ACM Trans. Program. Lang. Syst.*, 7(3):359–379, July 1985.
- [108] Jeanne Ferrante and Mary Mace. On linearizing parallel code. In *Proceedings of the 12th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, POPL ’85, page 179–190, New York, NY, USA, 1985. Association for Computing Machinery.
- [109] Paul Hudak and Benjamin Goldberg. Distributed execution of functional programs using serial combinators. *IEEE Transactions on computers*, 100(10):881–891, 1985.
- [110] Chris Clack and Simon L Peyton Jones. Strictness analysis—a practical approach. In *Conference on Functional Programming Languages and Computer Architecture*, pages 35–49. Springer, 1985.
- [111] D Harel. A linear algorithm for finding dominators in flow graphs and related problems. In *Proceedings of the Seventeenth Annual ACM Symposium on Theory of Computing*, STOC ’85, page 185–194, New York, NY, USA, 1985. Association for Computing Machinery.
- [112] David A. Padua and Michael J. Wolfe. Advanced compiler optimizations for supercomputers. *Commun. ACM*, 29(12):1184–1201, December 1986.

REFERENCES

- [113] Peter Harrison and Hessam Khoshnevisan. Efficient compilation of linear recursive functions into object level loops. In *Proceedings of the 1986 SIGPLAN Symposium on Compiler Construction*, SIGPLAN '86, page 207–218, New York, NY, USA, 1986. Association for Computing Machinery.
- [114] Keith D. Cooper, Ken Kennedy, and Linda Torczon. The impact of interprocedural analysis and optimization in the rn programming environment. *ACM Trans. Program. Lang. Syst.*, 8(4):491–523, August 1986.
- [115] Philip B Gibbons and Steven S Muchnick. Efficient instruction scheduling for a pipelined architecture. In *Proceedings of the 1986 SIGPLAN symposium on Compiler construction*, pages 11–16, 1986.
- [116] John H. Reif and Harry R. Lewis. Efficient symbolic analysis of programs. *Journal of Computer and System Sciences*, 32(3):280–314, 1986.
- [117] Randal E. Bryant. Graph-based algorithms for boolean function manipulation. *IEEE Transactions on Computers*, C-35(8):677–691, 1986.
- [118] Michael Wolfe. Loops skewing: The wavefront method revisited. *International Journal of Parallel Programming*, 15(4):279–293, 1986.
- [119] P YT Hsu and Edward S Davidson. Highly concurrent scalar processing. *ACM SIGARCH Computer Architecture News*, 14(2):386–395, 1986.
- [120] Robert Bernstein. Multiplication by integer constants. *Software: practice and experience*, 16(7):641–652, 1986.
- [121] David W. Wall. Global register allocation at link time. In *Proceedings of the 1986 SIGPLAN Symposium on Compiler Construction*, SIGPLAN '86, page 264–275, New York, NY, USA, 1986. Association for Computing Machinery.
- [122] David Callahan, Keith D Cooper, Ken Kennedy, and Linda Torczon. Interprocedural constant propagation. *ACM SIGPLAN Notices*, 21(7):152–161, 1986.
- [123] Keith D. Cooper, Ken Kennedy, and Linda Torczon. Interprocedural optimization: eliminating unnecessary recompilation. In *Proceedings of the 1986 SIGPLAN Symposium on Compiler Construction*, SIGPLAN '86, page 58–67, New York, NY, USA, 1986. Association for Computing Machinery.
- [124] Michael Burke and Ron Cytron. Interprocedural dependence analysis and parallelization. *ACM Sigplan Notices*, 21(7):162–175, 1986.
- [125] Mark S. Johnson and Terrence C. Miller. Effectiveness of a machine-level, global optimizer. In *Proceedings of the 1986 SIGPLAN Symposium on Compiler Construction*, SIGPLAN '86, page 99–108, New York, NY, USA, 1986. Association for Computing Machinery.
- [126] Rémi Triolet, Francois Irigoin, and Paul Feautrier. Direct parallelization of call statements. *SIGPLAN Not.*, 21(7):176–185, July 1986.
- [127] Ron Cytron, Andy Lowry, and F Kenneth Zadeck. Code motion of control structures in high-level languages. In *Proceedings of the 13th ACM SIGACT-SIGPLAN symposium on Principles of programming languages*, pages 70–85, 1986.
- [128] Jack W Davidson and Christopher W Fraser. Automatic inference and fast interpretation of peephole optimization rules. *Software: Practice and Experience*, 17(11):801–812, 1987.
- [129] Samuel P. Midkiff and David A. Padua. Compiler algorithms for synchronization. *IEEE Transactions on Computers*, C-36(12):1485–1495, 1987.
- [130] r. Allen, D. Callahan, and K. Kennedy. Automatic decomposition of scientific programs for parallel execution. In *Proceedings of the 14th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, POPL '87, page 63–76, New York, NY, USA, 1987. Association for Computing Machinery.
- [131] Constantine D. Polychronopoulos and David J. Kuck. Guided self-scheduling: A practical scheduling scheme for parallel supercomputers. *IEEE Transactions on Computers*, C-36(12):1425–1439, 1987.

REFERENCES

- [132] Constantine D Polychronopoulos. Loop coalescing: A compiler transformation for parallel machines. Technical report, Illinois Univ., Urbana (USA), 1987.
- [133] Dennis Gannon, William Jalby, and Kyle Gallivan. Strategies for cache and local memory management by global program transformation. In *International Conference on Supercomputing*, pages 229–254. Springer, 1987.
- [134] Jeanne Ferrante, Karl J. Ottenstein, and Joe D. Warren. The program dependence graph and its use in optimization. *ACM Trans. Program. Lang. Syst.*, 9(3):319–349, July 1987.
- [135] Randy Allen and Ken Kennedy. Automatic translation of fortran programs to vector form. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 9(4):491–542, 1987.
- [136] D. R. Chase. Safety consideration for storage allocation optimizations. In *Proceedings of the ACM SIGPLAN 1988 Conference on Programming Language Design and Implementation, PLDI ’88*, page 1–10, New York, NY, USA, 1988. Association for Computing Machinery.
- [137] Dennis Shasha and Marc Snir. Efficient and correct execution of parallel programs that share memory. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 10(2):282–312, 1988.
- [138] I. A. Natour. On the control dependence in the program dependence graph. In *Proceedings of the 1988 ACM Sixteenth Annual Conference on Computer Science, CSC ’88*, page 510–519, New York, NY, USA, 1988. Association for Computing Machinery.
- [139] D. E. Culler and Arvind. Resource requirements of dataflow programs. In *Proceedings of the 15th Annual International Symposium on Computer Architecture, ISCA ’88*, page 141–150, Washington, DC, USA, 1988. IEEE Computer Society Press.
- [140] M. D. Carroll and B. G. Ryder. Incremental data flow analysis via dominator and attribute update. In *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL ’88*, page 274–284, New York, NY, USA, 1988. Association for Computing Machinery.
- [141] S. Jain and C. Thompson. An efficient approach to data flow analysis in a multiple pass global optimizer. *SIGPLAN Not.*, 23(7):154–163, June 1988.
- [142] Karl-Heinz Drechsler and Manfred P. Stadel. A solution to a problem with morel and renvoise’s “global optimization by suppression of partial redundancies”. *ACM Trans. Program. Lang. Syst.*, 10(4):635–640, October 1988.
- [143] Keith D Cooper and Ken Kennedy. Efficient computation of flow-insensitive interprocedural summary information—a correction. *ACM SIGPLAN Notices*, 23(4):35–42, 1988.
- [144] D. R. Wallace. Dependence of multi-dimensional array references. In *Proceedings of the 2nd International Conference on Supercomputing, ICS ’88*, page 418–428, New York, NY, USA, 1988. Association for Computing Machinery.
- [145] T. Brandes. The importance of direct dependences for automatic parallelization. In *Proceedings of the 2nd International Conference on Supercomputing, ICS ’88*, page 407–417, New York, NY, USA, 1988. Association for Computing Machinery.
- [146] A. Lichnewsky and F. Thomasset. Introducing symbolic problem solving techniques in the dependence testing phases of a vectorizer. In *Proceedings of the 2nd International Conference on Supercomputing, ICS ’88*, page 396–406, New York, NY, USA, 1988. Association for Computing Machinery.
- [147] J. Ferrante, M. Mace, and B. Simons. Generating sequential code from parallel code. In *Proceedings of the 2nd International Conference on Supercomputing, ICS ’88*, page 582–592, New York, NY, USA, 1988. Association for Computing Machinery.
- [148] H. Dietz and C.-H. Chi. Cregs: a new kind of memory for referencing arrays and pointers. In *Supercomputing ’88: Proceedings of the 1988 ACM/IEEE Conference on Supercomputing, Vol. I*, pages 360–367, 1988.

REFERENCES

- [149] Constantine D. Polychronopoulos. Advanced loop optimizations for parallel computers. In E. N. Houstis, T. S. Papatheodorou, and C. D. Polychronopoulos, editors, *Supercomputing*, pages 255–277, Berlin, Heidelberg, 1988. Springer Berlin Heidelberg.
- [150] Utpal Banerjee. An introduction to a formal theory of dependence analysis. *The Journal of Supercomputing*, 2:133–149, 1988.
- [151] David Callahan and Ken Kennedy. Analysis of interprocedural side effects in a parallel programming environment. In E. N. Houstis, T. S. Papatheodorou, and C. D. Polychronopoulos, editors, *Supercomputing*, pages 138–171, Berlin, Heidelberg, 1988. Springer Berlin Heidelberg.
- [152] Paul Feautrier. Array expansion. In *ACM International Conference on Supercomputing 25th Anniversary Volume*, pages 99–111, 1988.
- [153] Alexandru Nicolau. Loop quantization: A generalized loop unwinding technique. *Journal of Parallel and Distributed Computing*, 5(5):568–586, 1988.
- [154] François Irigoin and Rémi Triolet. Supernode partitioning. In *Proceedings of the 15th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 319–329, 1988.
- [155] Dhananjay M Dhamdhere. A fast algorithm for code movement optimisation. *ACM SIGPLAN Notices*, 23(10):172–180, 1988.
- [156] David Callahan and Ken Kennedy. Compiling programs for distributed-memory multiprocessors. *The Journal of Supercomputing*, 2:151–169, 1988.
- [157] Alexander Aiken and Alexandru Nicolau. Optimal loop parallelization. *ACM SIGPLAN Notices*, 23(7):308–317, 1988.
- [158] M. Lam. Software pipelining: an effective scheduling technique for vliw machines. *SIGPLAN Not.*, 23(7):318–328, June 1988.
- [159] David Callahan. The program summary graph and flow-sensitive interprocedural data flow analysis. In *Proceedings of the ACM SIGPLAN 1988 conference on Programming Language design and Implementation*, pages 47–56, 1988.
- [160] Frances Allen, Michael Burke, Ron Cytron, Jeanne Ferrante, and Wilson Hsieh. A framework for determining useful parallelism. In *Proceedings of the 2nd international conference on Supercomputing*, pages 207–215, 1988.
- [161] Frances Allen, Michael Burke, Philippe Charles, Ron Cytron, and Jeanne Ferrante. An overview of the ptran analysis system for multiprocessing. *Journal of parallel and distributed computing*, 5(5):617–640, 1988.
- [162] Zhiyuan Li and Pen-Chung Yew. Efficient interprocedural analysis for program parallelization and restructuring. *SIGPLAN Not.*, 23(9):85–99, January 1988.
- [163] M. Girkar and C. Polychronopoulos. Compiling issues for supercomputers. In *Supercomputing '88: Proceedings of the 1988 ACM/IEEE Conference on Supercomputing, Vol. I*, pages 164–173, 1988.
- [164] Fred C Chow. Minimizing register usage penalty at procedure calls. In *Proceedings of the ACM SIGPLAN 1988 conference on Programming Language design and Implementation*, pages 85–94, 1988.
- [165] David Callahan, John Cocke, and Ken Kennedy. Estimating interlock and improving balance for pipelined architectures. *Journal of Parallel and Distributed Computing*, 5(4):334–358, 1988.
- [166] B. K. Rosen, M. N. Wegman, and F. K. Zadeck. Global value numbers and redundant computations. In *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '88*, page 12–27, New York, NY, USA, 1988. Association for Computing Machinery.
- [167] Alexander Aiken and Alexandru Nicolau. Perfect pipelining: A new loop parallelization technique. In *European Symposium on Programming*, pages 221–235. Springer, 1988.
- [168] James R Goodman and W-C Hsu. Code scheduling and register allocation in large basic blocks. In

REFERENCES

- ACM International Conference on Supercomputing 25th Anniversary Volume*, pages 88–98, 1988.
- [169] Keith D Cooper and Ken Kennedy. Interprocedural side-effect analysis in linear time. *ACM SIGPLAN Notices*, 23(7):57–66, 1988.
 - [170] V. Balasundaram and K. Kennedy. A technique for summarizing data access and its use in parallelism enhancing transformations. *SIGPLAN Not.*, 24(7):41–53, June 1989.
 - [171] Stephen Richardson and Mahadevan Ganapathi. Interprocedural analysis vs. procedure integration. *Information Processing Letters*, 32(3):137–142, 1989.
 - [172] C-H Chi and Hank Dietz. Unified management of registers and cache using liveness and cache bypass. In *Proceedings of the ACM SIGPLAN 1989 conference on Programming language design and implementation*, pages 344–353, 1989.
 - [173] Dhananjay M Dhamdhare. A new algorithm for composite hoisting and strength reduction optimization. *International Journal of Computer Mathematics*, 27(1):1–14, 1989.
 - [174] Mary Jean Harrold and Mary Lou Soffa. Interprocedural data flow testing. *SIGSOFT Softw. Eng. Notes*, 14(8):158–167, November 1989.
 - [175] S. McFarling. Program optimization for instruction caches. *SIGARCH Comput. Archit. News*, 17(2):183–191, April 1989.
 - [176] S. Horwitz, P. Pfeiffer, and T. Reps. Dependence analysis for pointer variables. *SIGPLAN Not.*, 24(7):28–40, June 1989.
 - [177] W. Baxter and H. R. Bauer. The program dependence graph and vectorization. In *Proceedings of the 16th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL ’89, page 1–11, New York, NY, USA, 1989. Association for Computing Machinery.
 - [178] W-m W Hwu and Pohua P Chang. Achieving high instruction cache performance with an optimizing compiler. In *Proceedings of the 16th Annual International Symposium on Computer Architecture*, pages 242–251, 1989.
 - [179] Susan J Eggers and Randy H Katz. Evaluating the performance of four snooping cache coherency protocols. In *Proceedings of the 16th annual international symposium on Computer architecture*, pages 2–15, 1989.
 - [180] Guy E Bluelloch. Scans as primitive parallel operations. *IEEE Transactions on computers*, 38(11):1526–1538, 1989.
 - [181] Alfred V. Aho, Mahadevan Ganapathi, and Steven W. K. Tjiang. Code generation using tree matching and dynamic programming. *ACM Trans. Program. Lang. Syst.*, 11(4):491–516, October 1989.
 - [182] R. Gupta, M. L. Soffa, and T. Steele. Register allocation via clique separators. In *Proceedings of the ACM SIGPLAN 1989 Conference on Programming Language Design and Implementation*, PLDI ’89, page 264–274, New York, NY, USA, 1989. Association for Computing Machinery.
 - [183] K. D. Cooper and K. Kennedy. Fast interprocedural alias analysis. In *Proceedings of the 16th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL ’89, page 49–59, New York, NY, USA, 1989. Association for Computing Machinery.
 - [184] P. Briggs, K. D. Cooper, K. Kennedy, and L. Torczon. Coloring heuristics for register allocation. In *Proceedings of the ACM SIGPLAN 1989 Conference on Programming Language Design and Implementation*, PLDI ’89, page 275–284, New York, NY, USA, 1989. Association for Computing Machinery.
 - [185] M. Wolfe. More iteration space tiling. In *Proceedings of the 1989 ACM/IEEE Conference on Supercomputing*, Supercomputing ’89, page 655–664, New York, NY, USA, 1989. Association for Computing Machinery.
 - [186] Zhiyuan Li, Pen-Chung Yew, and Chuag-Qi Zhu. Data dependence analysis on multi-dimensional array references. In *Proceedings of the 3rd International Conference on Supercomputing*, ICS ’89, page

REFERENCES

- 215–224, New York, NY, USA, 1989. Association for Computing Machinery.
- [187] D. Bernstein, M. Golumbic, y. Mansour, R. Pinter, D. Goldin, H. Krawczyk, and I. Nahshon. Spill code minimization techniques for optimizing compilers. In *Proceedings of the ACM SIGPLAN 1989 Conference on Programming Language Design and Implementation*, PLDI '89, page 258–263, New York, NY, USA, 1989. Association for Computing Machinery.
 - [188] Craig Chambers and David Ungar. Customization: Optimizing compiler technology for self, a dynamically-typed object-oriented programming language. *ACM SIGPLAN Notices*, 24(7):146–160, 1989.
 - [189] Ron Cytron, Jeanne Ferrante, Barry K Rosen, Mark N Wegman, and F Kenneth Zadeck. An efficient method of computing static single assignment form. In *Proceedings of the 16th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 25–35, 1989.
 - [190] Bernhard Steffen, Jens Knoop, and Oliver Rüthing. The value flow graph: A program representation for optimal program transformations. In Neil Jones, editor, *ESOP '90*, pages 389–405, Berlin, Heidelberg, 1990. Springer Berlin Heidelberg.
 - [191] David E. Hudak and Santosh G. Abraham. Compiler techniques for data partitioning of sequentially iterated parallel loops. In *Proceedings of the 4th International Conference on Supercomputing*, ICS '90, page 187–200, New York, NY, USA, 1990. Association for Computing Machinery.
 - [192] Lars Hernquist. Vectorization of tree traversals. *Journal of Computational Physics*, 87(1):137–147, 1990.
 - [193] R. Gupta. Loop displacement: an approach for transforming and scheduling loops for parallel execution. In *Supercomputing '90: Proceedings of the 1990 ACM/IEEE Conference on Supercomputing*, pages 388–397, 1990.
 - [194] Paul Havlak and Ken Kennedy. Experience with interprocedural analysis of array side effects. In *Proceedings of the 1990 ACM/IEEE Conference on Supercomputing*, Supercomputing '90, page 952–961, Washington, DC, USA, 1990. IEEE Computer Society Press.
 - [195] D. Whitfield and M. L. Soffa. An approach to ordering optimizing transformations. In *Proceedings of the Second ACM SIGPLAN Symposium on Principles & Practice of Parallel Programming*, PPOPP '90, page 137–146, New York, NY, USA, 1990. Association for Computing Machinery.
 - [196] Vatsa Santhanam and Daryl Odnert. Register allocation across procedure and module boundaries. *SIGPLAN Not.*, 25(6):28–39, June 1990.
 - [197] Rajiv Gupta and Mary Lou Soffa. Region scheduling: An approach for detecting and redistributing parallelism. *IEEE Trans. Softw. Eng.*, 16(4):421–431, April 1990.
 - [198] D. Callahan, A. Carle, M.W. Hall, and K. Kennedy. Constructing the procedure call multigraph. *IEEE Transactions on Software Engineering*, 16(4):483–487, 1990.
 - [199] David Klappholz, Kleanthis Psarris, and Xiangyun Kong. On the perfect accuracy of an approximate subscript analysis test. *ACM SIGARCH Computer Architecture News*, 18(3b):201–212, 1990.
 - [200] Thomas Gross and Peter Steenkiste. Structured dataflow analysis for arrays and its use in an optimizing compiler. *Software: Practice and Experience*, 20(2):133–155, 1990.
 - [201] Steven Anderson and Paul Hudak. Compilation of haskell array comprehensions for scientific computing. *ACM SIGPLAN Notices*, 25(6):137–149, 1990.
 - [202] William D. Clinger. How to read floating point numbers accurately. *SIGPLAN Not.*, 25(6):92–101, June 1990.
 - [203] Guy L. Steele and Jon L. White. How to print floating-point numbers accurately. *SIGPLAN Not.*, 25(6):112–126, June 1990.
 - [204] Karl Pettis and Robert C. Hansen. Profile guided code positioning. *SIGPLAN Not.*, 25(6):16–27, June

REFERENCES

- 1990.
- [205] Michael Gerndt. Updating distributed variables in local computations. *Concurrency: Practice and Experience*, 2(3):171–193, 1990.
 - [206] Michael Burke. An interval-based approach to exhaustive and incremental interprocedural data-flow analysis. *ACM Trans. Program. Lang. Syst.*, 12(3):341–395, July 1990.
 - [207] Fred C Chow and John L Hennessy. The priority-based coloring approach to register allocation. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 12(4):501–536, 1990.
 - [208] David Callahan, Steve Carr, and Ken Kennedy. Improving register allocation for subscripted variables. *ACM Sigplan Notices*, 25(6):53–65, 1990.
 - [209] David R. Chase, Mark Wegman, and F. Kenneth Zadeck. Analysis of pointers and structures. In *Proceedings of the ACM SIGPLAN 1990 Conference on Programming Language Design and Implementation, PLDI '90*, page 296–310, New York, NY, USA, 1990. Association for Computing Machinery.
 - [210] Ken Kennedy and Kathryn S McKinley. Loop distribution with arbitrary control flow. In *SC*, pages 407–416. Citeseer, 1990.
 - [211] Brian R. Nickerson. Graph coloring register allocation for processors with multi-register operands. *SIGPLAN Not.*, 25(6):40–52, June 1990.
 - [212] Michael E. Wolf and Monica S. Lam. A data locality optimizing algorithm. *SIGPLAN Not.*, 26(6):30–44, May 1991.
 - [213] Uwe Schwiegelshohn, Franco Gasperoni, and Kemal Ebcioglu. On optimal parallelization of arbitrary loops. *Journal of Parallel and Distributed Computing*, 11(2):130–134, 1991.
 - [214] Phil Pfeiffer and Rebecca Parsons Selke. On the adequacy of dependence-based representations for programs with heaps. In Takayasu Ito and Albert R. Meyer, editors, *Theoretical Aspects of Computer Software*, pages 365–386, Berlin, Heidelberg, 1991. Springer Berlin Heidelberg.
 - [215] Jong-Deok Choi, Ron Cytron, and Jeanne Ferrante. Automatic construction of sparse data flow evaluation graphs. In *Proceedings of the 18th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '91*, page 55–66, New York, NY, USA, 1991. Association for Computing Machinery.
 - [216] Seema Hiranandani, Ken Kennedy, and Chau-Wen Tseng. Compiler optimizations for fortran d on mimd distributed-memory machines. In *Supercomputing '91: Proceedings of the 1991 ACM/IEEE Conference on Supercomputing*, pages 86–100, 1991.
 - [217] David Callahan and Brian Koblenz. Register allocation via hierarchical graph coloring. *ACM Sigplan Notices*, 26(6):192–203, 1991.
 - [218] Suneel Jain. Circular scheduling: a new technique to perform software pipelining. In *Proceedings of the ACM SIGPLAN 1991 Conference on Programming Language Design and Implementation, PLDI '91*, page 219–228, New York, NY, USA, 1991. Association for Computing Machinery.
 - [219] Mark Smotherman, Sanjay Krishnamurthy, P. S. Aravind, and David Hunnicutt. Efficient dag construction and heuristic calculation for instruction scheduling. In *Proceedings of the 24th Annual International Symposium on Microarchitecture, MICRO 24*, page 93–102, New York, NY, USA, 1991. Association for Computing Machinery.
 - [220] Ron Cytron, Jeanne Ferrante, Barry K. Rosen, Mark N. Wegman, and F. Kenneth Zadeck. Efficiently computing static single assignment form and the control dependence graph. *ACM Trans. Program. Lang. Syst.*, 13(4):451–490, October 1991.
 - [221] David Callahan, Ken Kennedy, and Allan Porterfield. Software prefetching. In *Proceedings of the Fourth International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS IV*, page 40–52, New York, NY, USA, 1991. Association for Computing Machinery.

REFERENCES

- [222] C. Koelbel and P. Mehrotra. Compiling global name-space parallel loops for distributed execution. *IEEE Transactions on Parallel and Distributed Systems*, 2(4):440–451, 1991.
- [223] Paul Havlak and Ken Kennedy. An implementation of interprocedural bounded regular section analysis. *IEEE Transactions on Parallel & Distributed Systems*, 2(03):350–360, 1991.
- [224] David W. Wall. Limits of instruction-level parallelism. *SIGOPS Oper. Syst. Rev.*, 25(Special Issue):176–188, April 1991.
- [225] William Pugh. Uniform techniques for loop optimization. In *Proceedings of the 5th international conference on Supercomputing*, pages 341–352, 1991.
- [226] Mark N. Wegman and F. Kenneth Zadeck. Constant Propagation with Conditional Branches. *ACM Transactions on Programming Languages and Systems*, 13(2):181–210, April 1991.
- [227] Dror E. Maydan, John L. Hennessy, and Monica S. Lam. Efficient and exact data dependence analysis. *SIGPLAN Not.*, 26(6):1–14, May 1991.
- [228] Ron Cytron, Jeanne Ferrante, Barry K. Rosen, Mark N. Wegman, and F. Kenneth Zadeck. Efficiently computing static single assignment form and the control dependence graph. *ACM Trans. Program. Lang. Syst.*, 13(4):451–490, October 1991.
- [229] David Bernstein and Michael Rodeh. Global instruction scheduling for superscalar machines. *SIGPLAN Not.*, 26(6):241–255, May 1991.
- [230] Dhananjay M. Dhamdhare. Practical adaption of the global optimization algorithm of morel and renvoise. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 13(2):291–294, 1991.
- [231] Monica D. Lam, Edward E. Rothberg, and Michael E. Wolf. The cache performance and optimizations of blocked algorithms. In *Proceedings of the Fourth International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS IV*, page 63–74, New York, NY, USA, 1991. Association for Computing Machinery.
- [232] Michael E Wolf and Monica S Lam. A loop transformation theory and an algorithm to maximize parallelism. *IEEE Transactions on Parallel & Distributed Systems*, 2(04):452–471, 1991.
- [233] Paul Feautrier. Dataflow analysis of array and scalar references. *International Journal of Parallel Programming*, 20:23–53, 1991.
- [234] Vivek Sarkar and Guang R. Gao. Optimization of array accesses by collective loop transformations. In *Proceedings of the 5th International Conference on Supercomputing, ICS '91*, page 194–205, New York, NY, USA, 1991. Association for Computing Machinery.
- [235] Mary W Hall, Ken Kennedy, and Kathryn S McKinley. Interprocedural transformations for parallel code generation. In *Proceedings of the 1991 ACM/IEEE conference on Supercomputing*, pages 424–434, 1991.
- [236] Gina Goff, Ken Kennedy, and Chau-Wen Tseng. Practical dependence testing. *ACM SIGPLAN Notices*, 26(6):15–29, 1991.
- [237] Scott McFarling. Procedure merging with instruction caches. *ACM SIGPLAN Notices*, 26(6):71–79, 1991.
- [238] Keith D Cooper, Mary W Hall, and Linda Torczon. An experiment with inline substitution. *Software: Practice and Experience*, 21(6):581–601, 1991.
- [239] J. Ramanujam. Non-unimodular transformations of nested loops. In *Supercomputing '92: Proceedings of the 1992 ACM/IEEE Conference on Supercomputing*, pages 214–223, 1992.
- [240] Vivek Sarkar and Radhika Thekkath. A general framework for iteration-reordering loop transformations. In *Proceedings of the ACM SIGPLAN 1992 conference on Programming language design and implementation*, pages 175–187, 1992.
- [241] Wu Yang, Susan Horwitz, and Thomas Reps. A program integration algorithm that accommodates

REFERENCES

- semantics-preserving transformations. *ACM Trans. Softw. Eng. Methodol.*, 1(3):310–354, July 1992.
- [242] Michael Weiss. The transitive closure of control dependence: The iterated join. *ACM Letters on Programming Languages and Systems (LOPLAS)*, 1(2):178–190, 1992.
 - [243] Joseph Hummel, Laurie J. Hendren, and Alexandru Nicolau. A general data dependence test for dynamic, pointer-based data structures. In *Proceedings of the ACM SIGPLAN 1994 Conference on Programming Language Design and Implementation, PLDI '94*, page 218–229, New York, NY, USA, 1994. Association for Computing Machinery.
 - [244] Reinhard v. Hanxleden and Ken Kennedy. Relaxing simd control flow constraints using loop transformations. *SIGPLAN Not.*, 27(7):188–199, July 1992.
 - [245] Rajiv Gupta. Generalized dominators and post-dominators. In *Proceedings of the 19th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '92*, page 246–257, New York, NY, USA, 1992. Association for Computing Machinery.
 - [246] Yong-fong Lee and Barbara G Ryder. A comprehensive approach to parallel data flow analysis. In *Proceedings of the 6th International Conference on Supercomputing*, pages 236–247, 1992.
 - [247] S. Tjiang, M. Wolf, M. Lam, K. Pieper, and J. Hennessy. Integrating scalar optimization and parallelization. In Utpal Banerjee, David Gelernter, Alex Nicolau, and David Padua, editors, *Languages and Compilers for Parallel Computing*, pages 137–151, Berlin, Heidelberg, 1992. Springer Berlin Heidelberg.
 - [248] Steven W. K. Tjiang and John L. Hennessy. Sharlit—a tool for building optimizers. *SIGPLAN Not.*, 27(7):82–93, July 1992.
 - [249] Dhananjay M Dhamdhare, Barry K Rosen, and F Kenneth Zadeck. How to analyze large programs efficiently and informatively. In *Proceedings of the ACM SIGPLAN 1992 conference on Programming language design and implementation*, pages 212–223, 1992.
 - [250] S. A. Mahlke, W. Y. Chen, J. C. Gyllenhaal, and W.-M. W. Hwu. Compiler code transformations for superscalar-based high performance systems. In *Proceedings of the 1992 ACM/IEEE Conference on Supercomputing*, Supercomputing '92, page 808–817, Washington, DC, USA, 1992. IEEE Computer Society Press.
 - [251] Christopher W. Fraser, David R. Hanson, and Todd A. Proebsting. Engineering a simple, efficient code-generator generator. *ACM Lett. Program. Lang. Syst.*, 1(3):213–226, September 1992.
 - [252] Paul Feautrier. Some efficient solutions to the affine scheduling problem. i. one-dimensional time. *International journal of parallel programming*, 21:313–347, 1992.
 - [253] Frank Mueller and David B. Whalley. Avoiding unconditional jumps by code replication. In *Proceedings of the ACM SIGPLAN 1992 Conference on Programming Language Design and Implementation, PLDI '92*, page 322–330, New York, NY, USA, 1992. Association for Computing Machinery.
 - [254] Preston Briggs, Keith D. Cooper, and Linda Torczon. Rematerialization. *SIGPLAN Not.*, 27(7):311–321, July 1992.
 - [255] Preston Briggs, Keith D Cooper, and Linda Torczon. Coloring register pairs. *ACM Letters on Programming Languages and Systems (LOPLAS)*, 1(1):3–13, 1992.
 - [256] Ken Kennedy and Kathryn S McKinley. Optimizing for parallelism and data locality. In *Proceedings of the 6th international conference on Supercomputing*, pages 323–334, 1992.
 - [257] Ken Kennedy and Kathryn S McKinley. Maximizing loop parallelism and improving data locality via loop fusion and distribution. In *International Workshop on Languages and Compilers for Parallel Computing*, pages 301–320. Springer, 1993.
 - [258] Michael Wolfe. Beyond induction variables. *SIGPLAN Not.*, 27(7):162–174, July 1992.
 - [259] Mary W. Hall and Ken Kennedy. Efficient call graph analysis. *ACM Lett. Program. Lang. Syst.*,

REFERENCES

- 1(3):227–242, September 1992.
- [260] William Landi and Barbara G. Ryder. A safe approximate algorithm for interprocedural aliasing. In *Proceedings of the ACM SIGPLAN 1992 Conference on Programming Language Design and Implementation*, PLDI '92, page 235–248, New York, NY, USA, 1992. Association for Computing Machinery.
- [261] Keith D. Cooper, Mary W. Hall, and Linda Torczon. Unexpected side effects of inline substitution: a case study. *ACM Lett. Program. Lang. Syst.*, 1(1):22–32, March 1992.
- [262] William Pugh and David Wonnacott. Eliminating false data dependences using the omega test. *SIGPLAN Not.*, 27(7):140–151, July 1992.
- [263] Jens Knoop, Oliver Rüthing, and Bernhard Steffen. Lazy code motion. *SIGPLAN Not.*, 27(7):224–234, July 1992.
- [264] Anne Rogers and Kai Li. Software support for speculative loads. In *Proceedings of the Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS V, page 38–50, New York, NY, USA, 1992. Association for Computing Machinery.
- [265] William Pugh. A practical algorithm for exact array dependence analysis. *Commun. ACM*, 35(8):102–114, August 1992.
- [266] Zhiyuan Li. Array privatization for parallel execution of loops. In *Proceedings of the 6th International Conference on Supercomputing*, ICS '92, page 313–322, New York, NY, USA, 1992. Association for Computing Machinery.
- [267] Michael Wolfe, Chau-Wen Tseng, et al. The power test for data dependence. *IEEE Transactions on Parallel and Distributed Systems*, 3(5):591–601, 1992.
- [268] Torbjörn Granlund and Richard Kenner. Eliminating branches using a superoptimizer and the gnu c compiler. In *Proceedings of the ACM SIGPLAN 1992 conference on Programming language design and implementation*, pages 341–352, 1992.
- [269] Todd C. Mowry, Monica S. Lam, and Anoop Gupta. Design and evaluation of a compiler algorithm for prefetching. *SIGPLAN Not.*, 27(9):62–73, September 1992.
- [270] D. Maydan, S. Amarsinghe, and M. Lam. Data dependence and data-flow analysis of arrays. In Utpal Banerjee, David Gelernter, Alex Nicolau, and David Padua, editors, *Languages and Compilers for Parallel Computing*, pages 434–448, Berlin, Heidelberg, 1993. Springer Berlin Heidelberg.
- [271] A. J. C. Bik and H. A. G. Wijshoff. Advanced compiler optimizations for sparse computations. In *Proceedings of the 1993 ACM/IEEE Conference on Supercomputing*, Supercomputing '93, page 430–439, New York, NY, USA, 1993. Association for Computing Machinery.
- [272] Wei Li and Keshav Pingali. Access normalization: loop restructuring for numa computers. *ACM Trans. Comput. Syst.*, 11(4):353–375, November 1993.
- [273] Owen Kaser, C. R. Ramakrishnan, and Shaunak Pawagi. On the conversion of indirect to direct recursion. *ACM Lett. Program. Lang. Syst.*, 2(1–4):151–164, March 1993.
- [274] Evelyn Duesterwald, Rajiv Gupta, and Mary Lou Soffa. A practical data flow framework for array reference analysis and its use in optimizations. In *Proceedings of the ACM SIGPLAN 1993 Conference on Programming Language Design and Implementation*, PLDI '93, page 68–77, New York, NY, USA, 1993. Association for Computing Machinery.
- [275] David A. Barrett and Benjamin G. Zorn. Using lifetime predictors to improve memory allocation performance. *SIGPLAN Not.*, 28(6):187–196, June 1993.
- [276] M. Robert Ito and A. Zaafrani. Data flow analysis for parallel programs. In *Proceedings of the 1993 ACM Conference on Computer Science*, CSC '93, page 318–325, New York, NY, USA, 1993. Association for Computing Machinery.
- [277] Richard Johnson and Keshav Pingali. Dependence-based program analysis. In *Proceedings of the*

REFERENCES

- ACM SIGPLAN 1993 Conference on Programming Language Design and Implementation*, PLDI '93, page 78–89, New York, NY, USA, 1993. Association for Computing Machinery.
- [278] Karl-Heinz Drechsler and Manfred P Stadel. A variation of knoop, rüthing, and steffen's lazy code motion. *ACM SIGPLAN Notices*, 28(5):29–38, 1993.
 - [279] Jens Knoop. Lazy strength reduction. *International Journal of Programming Languages*, 1(1):71–91, 1993.
 - [280] G. Gao, R. Olsen, V. Sarkar, and R. Thekkath. Collective loop fusion for array contraction. In Utpal Banerjee, David Gelernter, Alex Nicolau, and David Padua, editors, *Languages and Compilers for Parallel Computing*, pages 281–295, Berlin, Heidelberg, 1993. Springer Berlin Heidelberg.
 - [281] Robert Metzger and Sean Stroud. Interprocedural constant propagation: an empirical study. *ACM Lett. Program. Lang. Syst.*, 2(1–4):213–232, March 1993.
 - [282] Mohammad R Haghighat and Constantine D Polychronopoulos. Symbolic analysis: A basis for parallelization, optimization, and scheduling of programs. In *International Workshop on Languages and Compilers for Parallel Computing*, pages 567–585. Springer, 1993.
 - [283] Dror E. Maydan, Saman P. Amarasinghe, and Monica S. Lam. Array-data flow analysis and its use in array privatization. In *Proceedings of the 20th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '93, page 2–15, New York, NY, USA, 1993. Association for Computing Machinery.
 - [284] Siddhartha Chatterjee, John R Gilbert, Robert Schreiber, and Shang-Hua Teng. Automatic array alignment in data-parallel programs. In *Proceedings of the 20th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 16–28, 1993.
 - [285] Jennifer M Anderson and Monica S Lam. Global optimizations for parallelism and locality on scalable parallel machines. *ACM Sigplan Notices*, 28(6):112–125, 1993.
 - [286] B. Ramakrishna Rau and Joseph A. Fisher. *Instruction-Level Parallel Processing: History, Overview, and Perspective*, pages 9–50. Springer US, Boston, MA, 1993.
 - [287] Amitabh Srivastava. A practical system for intermodule code optimization at link-time. *Journal of programming Languages*, 1(1):1–18, 1993.
 - [288] James R Larus. Loop-level parallelism in numeric and symbolic programs. *IEEE Transactions on Parallel and Distributed Systems*, 4(7):812–826, 1993.
 - [289] Susan L. Graham, Steven Lucco, and Oliver Sharp. Orchestrating interactions among parallel computations. *SIGPLAN Not.*, 28(6):100–111, June 1993.
 - [290] Wen-Mei W Hwu, Scott A Mahlke, William Y Chen, Pohua P Chang, Nancy J Warter, Roger A Bringmann, Roland G Ouellette, Richard E Hank, Tokuzo Kiyohara, Grant E Haab, et al. The superblock: an effective technique for vliw and superscalar compilation. *The Journal of Supercomputing*, 7(1–2):229–248, 1993.
 - [291] Rajiv Gupta. Optimizing array bound checks using flow analysis. *ACM Lett. Program. Lang. Syst.*, 2(1–4):135–150, March 1993.
 - [292] Shlomit S. Pinter. Register allocation with instruction scheduling. *SIGPLAN Not.*, 28(6):248–257, June 1993.
 - [293] Keith D Cooper, Mary W Hall, and Ken Kennedy. A methodology for procedure cloning. *Computer Languages*, 19(2):105–117, 1993.
 - [294] Jong-Deok Choi, Michael Burke, and Paul Carini. Efficient flow-sensitive interprocedural computation of pointer-induced aliases and side effects. In *Proceedings of the 20th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 232–245, 1993.
 - [295] Peng Tu and David Padua. Automatic array privatization. In *International Workshop on Languages*

REFERENCES

- and Compilers for Parallel Computing*, pages 500–521. Springer, 1993.
- [296] William Landi, Barbara G Ryder, and Sean Zhang. Interprocedural modification side effect analysis with pointer aliasing. *ACM SIGPLAN Notices*, 28(6):56–67, 1993.
 - [297] Thomas Ball and James R Larus. Branch prediction for free. *ACM SIGPLAN Notices*, 28(6):300–313, 1993.
 - [298] R. Sass and M. Mutka. Enabling unimodular transformations. In *Supercomputing '94: Proceedings of the 1994 ACM/IEEE Conference on Supercomputing*, pages 753–762, 1994.
 - [299] Steve Carr, Kathryn S. McKinley, and Chau-Wen Tseng. Compiler optimizations for improving data locality. *SIGPLAN Not.*, 29(11):252–262, November 1994.
 - [300] Lawrence Feigen, David Klappholz, Robert Casazza, and Xing Xue. The revival transformation. In *Proceedings of the 21st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '94*, page 421–434, New York, NY, USA, 1994. Association for Computing Machinery.
 - [301] William Blume and Rudolf Eigenmann. The range test: a dependence test for symbolic, non-linear expressions. In *Proceedings of the 1994 ACM/IEEE Conference on Supercomputing*, Supercomputing '94, page 528–537, Washington, DC, USA, 1994. IEEE Computer Society Press.
 - [302] Amitabh Srivastava and David W. Wall. Link-time optimization of address calculation on a 64-bit architecture. In *Proceedings of the ACM SIGPLAN 1994 Conference on Programming Language Design and Implementation, PLDI '94*, page 49–60, New York, NY, USA, 1994. Association for Computing Machinery.
 - [303] Preston Briggs and Keith D Cooper. Effective partial redundancy elimination. *ACM SIGPLAN Notices*, 29(6):159–170, 1994.
 - [304] Jens Knoop, Oliver Rüthing, and Bernhard Steffen. Partial dead code elimination. *ACM Sigplan Notices*, 29(6):147–158, 1994.
 - [305] Uma Mahadevan and Sridhar Ramakrishnan. Instruction scheduling over regions: A framework for scheduling across basic blocks. In Peter A. Fritzson, editor, *Compiler Construction*, pages 419–434, Berlin, Heidelberg, 1994. Springer Berlin Heidelberg.
 - [306] Daniel Weise, Roger F. Crew, Michael Ernst, and Bjarne Steensgaard. Value dependence graphs: representation without taxation. In *Proceedings of the 21st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '94*, page 297–310, New York, NY, USA, 1994. Association for Computing Machinery.
 - [307] Steven M. Kurlander and Charles N. Fischer. Zero-cost range splitting. *SIGPLAN Not.*, 29(6):257–265, June 1994.
 - [308] Brad Calder and Dirk Grunwald. Reducing branch costs via branch alignment. *SIGOPS Oper. Syst. Rev.*, 28(5):242–251, November 1994.
 - [309] Steve Carr and Ken Kennedy. Improving the ratio of memory operations to floating-point operations in loops. *ACM Trans. Program. Lang. Syst.*, 16(6):1768–1810, November 1994.
 - [310] Eric Amiel, Olivier Gruber, and Eric Simon. Optimizing multi-method dispatch using compressed dispatch tables. *SIGPLAN Not.*, 29(10):244–258, October 1994.
 - [311] Cliff Young and Michael D. Smith. Improving the accuracy of static branch prediction using branch correlation. *SIGOPS Oper. Syst. Rev.*, 28(5):232–241, November 1994.
 - [312] David F. Bacon, Jyh-Herng Chow, Dz-Ching Ju, Kalyan Muthukumar, and Vivek Sarkar. A compiler framework for restructuring data declarations to enhance cache and TLB effectiveness. In John E. Botsford, Ann Gawman, W. Morven Gentleman, Evelyn Kidd, Kelly A. Lyons, Jacob Slonim, and J. Howard Johnson, editors, *Proceedings of the 1994 Conference of the Centre for Advanced Studies on Collaborative Research, October 31 - November 3, 1994, Toronto, Ontario, Canada*, page 3. IBM, 1994.

REFERENCES

- [313] J. Torrellas, H.S. Lam, and J.L. Hennessy. False sharing and spatial locality in multiprocessor caches. *IEEE Transactions on Computers*, 43(6):651–663, 1994.
- [314] Peter W Markstein, Victoria Markstein, and F Kenneth Zadeck. Reassociation and strength reduction. *Optimization in Compilers*, 1994.
- [315] Siddhartha Chatterjee, John R Gilbert, and Robert Schreiber. The alignment-distribution graph. In *Languages and Compilers for Parallel Computing: 6th International Workshop Portland, Oregon, USA, August 12–14, 1993 Proceedings* 6, pages 234–252. Springer, 1994.
- [316] Preston Briggs, Keith D. Cooper, and Linda Torczon. Improvements to graph coloring register allocation. *ACM Trans. Program. Lang. Syst.*, 16(3):428–455, May 1994.
- [317] Maryam Emami, Rakesh Ghiya, and Laurie J. Hendren. Context-sensitive interprocedural points-to analysis in the presence of function pointers. In *Proceedings of the ACM SIGPLAN 1994 Conference on Programming Language Design and Implementation*, PLDI '94, page 242–256, New York, NY, USA, 1994. Association for Computing Machinery.
- [318] Alain Deutsch. Interprocedural may-alias analysis for pointers: beyond k-limiting. In *Proceedings of the ACM SIGPLAN 1994 Conference on Programming Language Design and Implementation*, PLDI '94, page 230–241, New York, NY, USA, 1994. Association for Computing Machinery.
- [319] Steve Carr and Ken Kennedy. Scalar replacement in the presence of conditional control flow. *Softw. Pract. Exper.*, 24(1):51–77, January 1994.